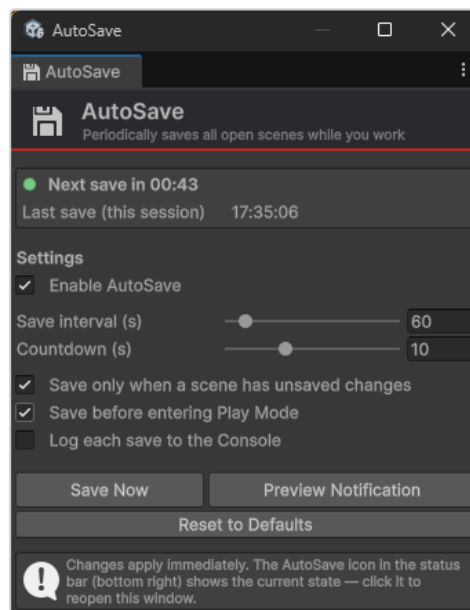

AutoSave

Automatic scene saving for the Unity Editor



Version 1.0.0

Unity 2021.3 LTS or newer

Editor-only — no runtime code, no build footprint

Ahmed Afifi

elasticated.dev

Table of Contents

1. Introduction

- 1.1 What AutoSave Does
- 1.2 Requirements
- 1.3 Package Contents

2. Installation

- 2.1 Importing the Package
- 2.2 Verifying the Installation

3. Setup Guide

- 3.1 Step 1 — Open the Settings Window
- 3.2 Step 2 — Enable AutoSave
- 3.3 Step 3 — Choose a Save Interval
- 3.4 Step 4 — Choose a Countdown Duration
- 3.5 Step 5 — Decide When Saves Happen
- 3.6 Step 6 — Enable Console Logging (Optional)
- 3.7 Step 7 — Confirm It Works

4. Daily Use

- 4.1 The Status Bar Icon
- 4.2 The Countdown Toast
- 4.3 Window Actions

5. How It Works

- 5.1 The Save Cycle
- 5.2 Skip Conditions
- 5.3 Domain Reload Safety

6. Script Reference

- 6.1 Namespace
- 6.2 AutoSave
- 6.3 AutoSave.SaveState
- 6.4 AutoSaveWindow
- 6.5 AutoSaveNotificationWindow
- 6.6 AutoSaveStatusBarIcon
- 6.7 EditorPrefs Keys
- 6.8 Example — Custom Save Trigger

7. Troubleshooting

8. Support

1. Introduction

1.1 What AutoSave Does

AutoSave saves every open scene at a fixed interval while you work in the Unity Editor, so an editor crash, a power cut, or an accidental Play Mode entry never costs you more than one interval of work.

Three parts make up the package:

- A background controller that runs the timer and performs the saves.
- A settings window where every option is configured. Changes apply immediately.
- A status bar icon that shows the current state at a glance and opens the settings on click.

Before each save, an optional countdown toast appears above the status bar. It gives you a few seconds to cancel the save, or to confirm it early. If you set the countdown to zero, saves happen silently.

1.2 Requirements

Item	Requirement
Unity version	2021.3 LTS or newer
Render pipeline	Any — the package does not render anything
Dependencies	None. Only built-in Unity Editor APIs are used.
Platforms	Editor-only. No code is included in a player build.

Note: Every script lives in an `Editor` folder, so Unity excludes the package from all builds automatically. AutoSave adds nothing to your build size.

1.3 Package Contents

File	Purpose
<code>AutoSave.cs</code>	Timer, save logic, and settings storage.
<code>AutoSaveWindow.cs</code>	The settings window.
<code>AutoSaveNotificationWindow.cs</code>	The countdown toast.
<code>AutoSaveStatusBarIcon.cs</code>	The status bar icon.
<code>AutoSave-Documentation.pdf</code>	This document.

2. Installation

2.1 Importing the Package

1. Open the project you want to add AutoSave to.
2. Select `Window ▶ Package Manager` , then choose `My Assets` .
3. Find **AutoSave** in the list and select `Download` , then `Import` .
4. Leave every file selected in the import dialog and select `Import` .

Unity compiles the scripts as soon as the import finishes. No further setup is required.

2.2 Verifying the Installation

AutoSave is installed correctly when both of the following are true:

1. The menu item `Window ▶ AutoSave ▶ Settings` exists.
2. A save icon appears in the status bar at the bottom right of the editor window.

AutoSave is enabled by default with a 60 second interval, so it starts protecting your work the moment the import completes.

3. Setup Guide

This section walks through every setting in order. The defaults suit most projects, so you can stop after step 2 if you do not need to change anything.

3.1 Step 1 — Open the Settings Window

Select `Window ▶ AutoSave ▶ Settings` , or click the save icon in the status bar at the bottom right of the editor. The window can be docked like any other Unity window.

The status card at the top of the window shows the current state and a live countdown to the next save, along with the time of the last successful save in this editor session.

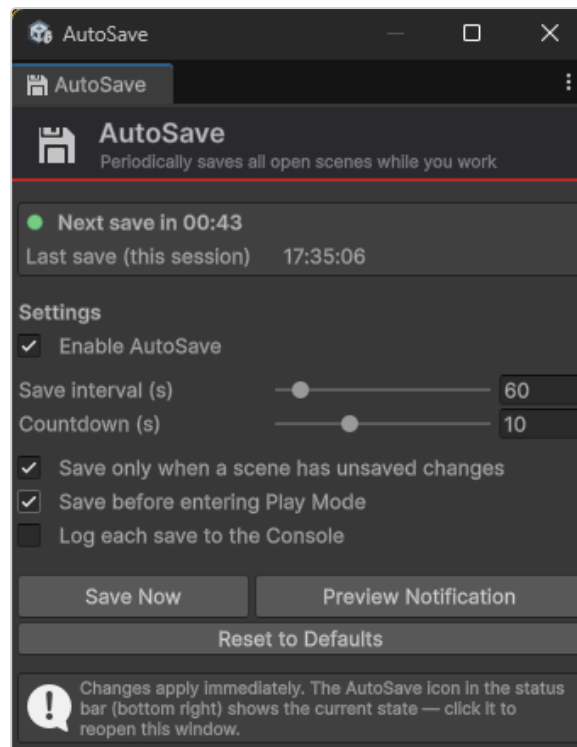


Figure 1 — The AutoSave settings window. Every option is on one page, and changes apply immediately.

3.2 Step 2 — Enable AutoSave

Enable AutoSave is the master switch. When it is off, no timer runs and no saves occur. It is on by default. Every other setting below is disabled while the master switch is off.

3.3 Step 3 — Choose a Save Interval

Save interval (s) controls how often AutoSave triggers. The range is 10 to 600 seconds and the default is 60.

Choose the interval by asking how much work you are willing to lose. A 60 second interval means you never lose more than a minute. Very large scenes take longer to write to disk, so raise the interval if you notice a pause on each save.

3.4 Step 4 — Choose a Countdown Duration

Countdown (s) controls the warning toast that appears before each save. The range is 0 to 30 seconds and the default is 5.

Value	Behaviour
0	No toast. Scenes are saved instantly and silently.
1 to 30	A toast appears for that many seconds. You can cancel or confirm the save.

A countdown is useful when you sometimes make changes you do not want written to disk, because it gives you a moment to cancel.

3.5 Step 5 — Decide When Saves Happen

Two toggles narrow down when a save is allowed:

Setting	Default	Effect
Save only when a scene has unsaved changes	On	Skips the save if no open scene is dirty. Recommended, as it avoids pointless disk writes and version control noise.
Save before entering Play Mode	On	Saves all open scenes the moment you press Play. This protects against an editor crash during Play Mode.

3.6 Step 6 — Enable Console Logging (Optional)

Log each save to the Console prints a line every time AutoSave writes your scenes. It is off by default so that a frequent interval never floods the Console. Turn it on when you want to confirm that saves are happening, then turn it off again.

3.7 Step 7 — Confirm It Works

1. Set **Countdown (s)** to 5 or higher.
2. Select **Preview Notification** at the bottom of the window.
3. The countdown toast appears above the status bar. Let it finish to save, or select X to cancel.

Preview Notification ignores the dirty-scene check, so the toast always appears even when nothing has changed. This makes it a reliable way to confirm the setup.

4. Daily Use

After setup, AutoSave needs no attention. This section describes what you will see while working.

4.1 The Status Bar Icon

The icon sits at the bottom right of the editor, to the left of Unity's own status bar icons. Hover over it for a tooltip with the time until the next save and the time of the last save. Click it to open the settings window.



Figure 2 — The AutoSave icon (left) sits flush against Unity's own status bar icons. Here it is red, meaning a save is counting down.

Appearance	Meaning
White	Enabled and waiting for the next save.
Red	A save is counting down right now.

Dimmed	AutoSave is disabled.
--------	-----------------------

4.2 The Countdown Toast

The toast appears above the status bar and shows the seconds remaining and a progress bar. It never takes keyboard focus, so it cannot interrupt your typing.

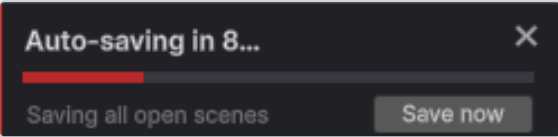


Figure 3 — The countdown toast. Select X to skip this save, or **Save now** to save without waiting.

Action	Result
Wait for the countdown to reach zero	All open scenes are saved.
Select Save now	Saves immediately without waiting.
Select X	Skips this save. The timer restarts.

4.3 Window Actions

Button	Result
Save Now	Saves all open scenes immediately and restarts the timer.
Preview Notification	Shows the countdown toast right away, ignoring the dirty-scene check.
Reset to Defaults	Restores every setting to its original value.

5. How It Works

5.1 The Save Cycle

1. The controller subscribes to `EditorApplication.update` when the editor loads.
2. Each tick it compares `EditorApplication.timeSinceStartup` against the next save time.
3. When the interval elapses, the skip conditions in section 5.2 are evaluated.
4. If no condition applies and the countdown is above zero, the toast opens. Otherwise scenes are saved at once.
5. After a save or a cancellation, the timer restarts from the full interval.

Saving is performed by `EditorSceneManager.SaveOpenScenes()`, which writes every open scene. The time of a successful save is recorded in `SessionState`, so it survives a script recompile but resets when the editor closes.

5.2 Skip Conditions

A save is skipped, and the timer restarted, in any of these cases:

Condition	Reason
AutoSave is disabled	The master switch is off.
The editor is compiling or importing	A domain reload is imminent and would orphan the toast.
Play Mode is active or starting	Scene changes during Play Mode are temporary and must not be written to disk.
No open scene is dirty	Only when <i>Save only when a scene has unsaved changes</i> is on.

Important: AutoSave never saves during Play Mode. Changes you make to a scene while playing are discarded by Unity when you exit Play Mode, and writing them to disk would be wrong.

5.3 Domain Reload Safety

A borderless editor window that survives a domain reload becomes unclickable, because the callbacks it held are gone. AutoSave prevents this in three ways:

1. The toast is closed before every assembly reload, via `AssemblyReloadEvents.beforeAssemblyReload`.
2. A toast that is deserialized after a reload detects that it was not created normally and closes itself.
3. A hard deadline in the update tick force-closes any toast that outlives its countdown by more than ten seconds.

All state in the package is editor state, held by static fields on classes marked `[InitializeOnLoad]`. No runtime script is included, so the *Fast Enter Play Mode* setting introduced in Unity 6.6 — which preserves static values and static event subscribers between Play Mode sessions — has no effect on this package. Entering Play Mode with a reload disabled leaves the editor callbacks already subscribed and the timer already running, which is the intended behaviour.

6. Script Reference

AutoSave works entirely through its settings window. This reference is for the rarer case where you want to drive it from your own editor tooling.

6.1 Namespace

Every type in the package lives in a single namespace:

```
using Elasticated.SceneAutoSave;
```

Note: Because the package is editor-only, any script that calls into it must live in an `Editor` folder, or in an assembly definition whose platform list contains only `Editor`.

6.2 AutoSave

```
[InitializeOnLoad] public static class AutoSave
```

The controller. It loads its settings and starts its timer when the editor loads, so you never construct or initialise it yourself.

Properties

Signature	Description
<code>static bool Enabled</code>	Whether the master switch is on.
<code>static double Interval</code>	Seconds between save attempts. Never below 10.
<code>static bool OnlyIfEdited</code>	Whether saves are skipped when no scene is dirty.
<code>static bool BeforePlayMode</code>	Whether scenes are saved when Play Mode starts.
<code>static double CountdownDuration</code>	Toast duration in seconds. Zero means no toast.
<code>static bool VerboseLogging</code>	Whether each save prints a line to the Console.
<code>static SaveState State</code>	The current state. See section 6.3.
<code>static double SecondsUntilNextSave</code>	Seconds until the next attempt. Zero while counting down.
<code>static DateTime? LastSaveTime</code>	Time of the last successful save this session, or <code>null</code> if there has not been one.

Methods

Signature	Description
<code>static void SaveNow()</code>	Cancels any countdown, saves all open scenes, and restarts the timer.
<code>static void CancelCountdown()</code>	Closes the toast without saving and restarts the timer.
<code>static void TriggerCountdownNow()</code>	Starts the countdown immediately, ignoring the dirty-scene check. Does nothing if a countdown is already running or the countdown duration is zero.
<code>static void ReloadSettings()</code>	Re-reads every setting from <code>EditorPrefs</code> and restarts the timer. Call this after writing a key yourself.

6.3 AutoSave.SaveState

```
public enum SaveState
```

Member	Meaning
Disabled	The master switch is off. No timer is running.
Waiting	Enabled and waiting for the interval to elapse.
CountingDown	The toast is open and a save is pending.

6.4 AutoSaveWindow

```
public class AutoSaveWindow : EditorWindow
```

The settings window, reachable at `Window ▶ AutoSave ▶ Settings`.

Signature	Description
<code>static void ShowWindow()</code>	Opens or focuses the window. Also the target of the menu item and the status bar icon.

6.5 AutoSaveNotificationWindow

```
public class AutoSaveNotificationWindow : EditorWindow
```

The countdown toast. The controller owns its lifetime, so you would not normally open one yourself.

Signature	Description
<code>static AutoSaveNotificationWindow Open(double countdownSeconds, Action onConfirm, Action onCancel)</code>	Opens a toast anchored above the status bar. <code>onConfirm</code> runs if the countdown completes or the user selects Save now ; <code>onCancel</code> runs if the user dismisses it.
<code>void CloseSilently()</code>	Closes the toast without firing either callback.
<code>static void CloseStrayInstances()</code>	Destroys toast windows left behind by a domain reload.

6.6 AutoSaveStatusBarIcon

```
[InitializeOnLoad] public static class AutoSaveStatusBarIcon
```

Injects the status bar icon and refreshes its colour and tooltip twice per second. It reaches Unity's status bar through internal editor APIs, so it is written defensively: if a future Unity version changes those APIs, the injection fails quietly and the menu item continues to work.

Signature	Description
<code>const float RightOffset</code>	Distance in pixels from the right edge of the status bar to the icon. Kept in sync with the toast's anchor.

6.7 EditorPrefs Keys

Settings are stored per user in `EditorPrefs`, not in the project, so each team member keeps their own preferences and nothing is committed to version control.

Constant	Key	Type	Default
<code>AutoSave.KeyEnabled</code>	<code>AutoSaveEnabled</code>	bool	<code>true</code>
<code>AutoSave.KeyInterval</code>	<code>AutoSaveInterval</code>	float	<code>60</code>
<code>AutoSave.KeyOnlyIfEdited</code>	<code>SaveOnlyIfEdited</code>	bool	<code>true</code>
<code>AutoSave.KeyBeforePlayMode</code>	<code>SaveBeforePlayMode</code>	bool	<code>true</code>
<code>AutoSave.KeyCountdown</code>	<code>AutoSaveCountdown</code>	float	<code>5</code>
<code>AutoSave.KeyVerboseLogging</code>	<code>AutoSaveVerboseLogging</code>	bool	<code>false</code>

6.8 Example — Custom Save Trigger

The example below adds a menu item that saves immediately, and a second one that raises the interval to five minutes. Note the `ReloadSettings()` call: writing a key does not by itself update the running timer.

```
using Elasticated.SceneAutoSave;
using UnityEditor;

namespace MyStudio.EditorTools
{
    public static class AutoSaveShortcuts
    {
        [MenuItem("Tools/My Studio/Save Scenes Now")]
        private static void SaveNow() => AutoSave.SaveNow();

        [MenuItem("Tools/My Studio/Use A Five Minute Interval")]
        private static void UseLongInterval()
        {
            EditorPrefs.SetFloat(AutoSave.KeyInterval, 300f);
            AutoSave.ReloadSettings();
        }
    }
}
```

7. Troubleshooting

The status bar icon does not appear

The icon is added through internal editor APIs, and a Unity version may change them. Use `Window ▶ AutoSave ▶ Settings` instead; every feature is reachable from the window. Please report the Unity version so a fix can be released.

My scenes are not being saved

Check each of the following in order:

1. Open the settings window and confirm the status card does not read *Disabled*.
2. Confirm you are not in Play Mode. AutoSave never saves while the game is playing.
3. If *Save only when a scene has unsaved changes* is on, confirm a scene is actually dirty. Unity marks a dirty scene with an asterisk in the Hierarchy.
4. Turn on *Log each save to the Console* and wait for one interval to see whether a save is reported.

The countdown toast is in the wrong place

The toast is anchored to the main editor window. If you moved the editor between monitors with different scaling, close the toast and let the next countdown reopen it at the corrected position.

Saving causes a noticeable pause

The pause is Unity writing the scene, not AutoSave. Large scenes take longer. Raise the save interval, or split large scenes into additively loaded ones.

The Console is full of AutoSave messages

Turn off *Log each save to the Console* in the settings window. It is off by default.

8. Support

Please include your Unity version, your operating system, and the steps to reproduce the problem. This makes a fix far quicker.

Channel	Address
Email	aafifi1988@yahoo.com
Website	https://elasticated.dev